Reviewer Pack — Minimal Geometric Entropy Bound for Circuit Entanglement Com…

Entry 7983dea1c76c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 18:12:01 UTC

Minimal Geometric Entropy Bound for Circuit Entanglement Complexity

Entry ID: `7983dea1c76c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 18:12:01 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Geometric Entropy) **Field B** (complexity object): Boolean Circuits (Circuit Entanglement Complexity)

Statement:

> For every d-regular Boolean circuit C with n inputs, the geometric entropy H(ρ_C) of its density matrix ρ_C is bounded by the entanglement complexity E(C), such that H(ρ_C) = O(E(C)) and H(ρ_C) ≤ 1.5 * E(C).

Rationale (proposer's reasoning):

> Geometric entropy provides a measure of quantum entanglement, and it has been used in other complexity measures like communication complexity. This conjecture aims to bridge this concept with circuit entanglement complexity, which could expose new insights into the structure of Boolean circuits.

Taxonomy category: `Quantum_Information_Theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `39dc4c85250fd4a4`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each d-regular Boolean circuit C, if the geometric entropy H(ρ_C) is bounded by O(E(C)) and H(ρ_C) ≤ 1.5 * E(C), where E(C) is the entanglement complexity calculated from the number of commuting pairs in the output of C.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `geometric entropy AND circuit entanglement complexity` - `Boolean circuits AND entanglement complexity AND geometric entropy bound` - `quantum information theory AND geometric entropy AND Boolean circuit complexity`

Top relevant hits considered: - [http://arxiv.org/abs/1903.03401v4] Planar Black holes and Entanglement Entropy in Analog Gravity Models - [http://arxiv.org/abs/1511.02288v2] Thermodynamic law from the entanglement entropy bound - [http://arxiv.org/abs/1511.02028v3] Hagedorn transition and topological entanglement entropy - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/2002.12814v2] On estimating the entropy of shallow circuit outputs - [http://arxiv.org/abs/2111.08018v2] Entanglement dynamics in hybrid quantum circuits - [http://arxiv.org/abs/1705.00365v2] Measuring Holographic Entanglement Entropy on a Quantum Simulator - [http://arxiv.org/abs/quant-ph/0305192v1] Photon engineering for quantum information processing

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def d_regular_circuit(n, d):
        if n % d != 0:
            return None
        circuit = []
        for i in range(n):
            row = [random.choice([0, 1]) for _ in range(d)]
            circuit.append(row)
        return circuit

    def density_matrix(circuit):
        n = len(circuit)
        rho = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i, n):
                count = 0
                for k in range(2**n):
                    input_bits = [k >> (i + j) & 1 for i in range(j - i + 1)]
                    if circuit[i][input_bits[0]] == circuit[j][input_bits[-1]]:
```

```
                    count += 1
            rho[i][j] = count / 2**n
            rho[j][i] = rho[i][j]
    return rho

def geometric_entropy(rho):
    n = len(rho)
    entropy = 0
    for i in range(n):
        if rho[i][i] > 0:
            entropy -= rho[i][i] * math.log2(rho[i][i])
    return entropy

def entanglement_complexity(circuit):
    n = len(circuit)
    commuting_pairs = 0
    for i in range(n):
        for j in range(i + 1, n):
            commute = True
            for k in range(2**n):
                input_bits = [k >> (i + j) & 1 for i in range(j - i + 1)]
                if circuit[i][input_bits[0]] != circuit[j][input_bits[-1]]:
                    commute = False
                    break
            if commute:
                commuting_pairs += 1
    return commuting_pairs

n_values = [5, 10, 15, 20, 30, 40]
instances_tested = 0
total_entropy = 0.0
max_n = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5):
        circuit = d_regular_circuit(n, 2)
        if circuit is None:
            continue
        rho = density_matrix(circuit)
        entropy = geometric_entropy(rho)
        E_C = entanglement_complexity(circuit)

        instances_tested += 1
        max_n = max(max_n, n)

        total_entropy += entropy

        if entropy > 1.5 * E_C:
            conjecture_holds = False
            counterexample = f"n={n}, E(C)={E_C}, H(ρ_C)={entropy}"

mean_entropy = total_entropy / instances_tested

return {
```

```
        "metric_name": "geometric_entropy",
        "metric_value": mean_entropy,
        "instances_tested": instances_tested,
        "n_max": max_n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14266
2	preregistration	ollama_remote	glm4:latest	0	0	9952
3	novelty	ollama_remote	glm4:latest	0	0	8288
4	novelty	ollama_remote	glm4:latest	0	0	9622
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30377
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14656
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11705
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	50558
9	judge	ollama_remote	glm4:latest	0	0	67029

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 216453 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7983dea1c76c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7983dea1c76c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7983dea1c76c.tar.gz` (if generated)